# Transaction and Communication Management System

Project Overview and API Design

# Overview

- Comprehensive platform to manage users, transactions, reviews, products, and communication.
- - Uses Firebase Firestore as database.
- - Joi for input validation.
- - Role-based access control for secure feature access.

# Features (Users & Transactions)

- Users:
- - Create and update user info.

- Transactions:
- - Create and update transactions.
- - Reorder alerts for vendors based on stock levels.

# Features (Reviews, Products, Messages)

- Reviews:
- - Add/update reviews for vendors/products.

- Products:
- - Add/update product details.
- - Stock management and reorder alerts.

- Messages:
- - Send/update/delete messages.

# Technologies Used

- - Backend: Node.js + Express
- - Database: Firebase Firestore
- - Validation: Joi
- - Authentication: JWT
- - Role-Based Access Control
- - Custom message model

# Installation and Setup

- 1. Clone the repository
- 2. Install dependencies (npm install)
- 3. Configure Firebase in firebaseConfig.js
- 4. Start application (npm start)
- Use Postman to test APIs.

# API Endpoints - Users & Transactions

- Users:
- POST /users
- PUT /users/:id

- Transactions:
- POST /transactions
- PUT /transactions/:id

# API Endpoints - Reviews & Products

- Reviews:
- POST /reviews
- PUT /reviews/:id

- Products:
- POST /products
- PUT /products/:id

# API Endpoints - Messages

- POST /messages
- PUT /messages/:id
- PATCH /messages/:id/markAsRead
- DELETE /messages/:id

# Testing

- - Use Postman to test endpoints.
- - Include headers for authentication (JWT tokens).

# Contributing

- 1. Fork the repo
- 2. Create a branch
- 3. Commit and push changes
- 4. Open a Pull Request

# License

- This project is licensed under the MIT License. See LICENSE file for details.